COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	October 13, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Asajar, Carlo Mari Godino, Jaime Jr.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 ALGORITHM STRATEGIES

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

For the pre-lab review, you must fill out the table below. Using algorithms implemented in the previous activities, identify which algorithm satisfies the given algorithmic strategy. For the analysis, you must explicitly provide what part of the algorithm you have identified matches the corresponding strategy.

Complete the table.
(Table 12-1. Algorithmic Strategies and Example)

Strategy	Algorithm	Analysis
Recursion	**Quick Sort**	Quick Sort uses recursion by choosing a pivot and separating the smaller and larger values into different parts. After separating the values, the same function is called again to sort each part until only one or no element is left. This process keeps repeating until all the values are arranged in the correct order.
Brute Force	**Bubble Sort**	Bubble Sort uses brute force by comparing two elements beside each other and swapping them when they are in the wrong order. The same process is repeated through several passes until the values are sorted properly. Even though many comparisons are needed, the algorithm keeps checking the elements and fixing their order until the sorting is completed.
Backtracking	**Depth First Searching**	Depth First Search (DFS) uses backtracking by following one path as far as possible before going back to a previous point when there are no more unvisited neighboring points to explore. After returning, the search continues through another available path. This process keeps repeating until the needed points are visited or there are no more paths left to explore.
Greedy	**Selection**	Selection Sort follows a greedy-like strategy by choosing the smallest element

	Sort	from the unsorted part during each pass. The selected element is then placed in its correct position before moving to the next part. By choosing the smallest value each time, the algorithm slowly arranges the elements in the correct order until the whole list becomes sorted.
Divide-and-Conquer	**Merge Sort**	Merge Sort uses divide-and-conquer by dividing the list into smaller halves until each part contains only one or no element. After dividing the list, the smaller parts are sorted and combined in the correct order. This process continues until all the parts are merged together and the whole list becomes sorted properly.

Table 12-1. Algorithmic Strategies and Examples

The Problem: MINIMUM STEPS TO ONE

Memoization

Run this code and show the output in the table.
(Table 12-2. Memoization Implementation)

Screenshot of Code	**My Code: (memoization_ILOA.cpp)**
	ILOA > memoization_ILOA.cpp > main()
	`#include <iostream>`
	`#include <vector>`
	`#include <chrono>`
	`#include <algorithm>`
	
	`// Store the previously calculated results`
	`std::vector<int> memo;`
	
	`// Find the minimum steps to reach 1`
	`int getMinStepsMemo(int n){`
	
	`    // Check if the number reaches 1`
	`    if(n == 1){`
	`        return 0;`
	`    }`
	
	`    // Check if the result is already stored`
	`    if(memo[n] != -1){`
	`        return memo[n];`

```
    }

    // Calculate the steps by subtracting 1
    int r = 1 + getMinStepsMemo(n - 1); // '-1' step

    // Check if the number can be divided by 2
    if(n % 2 == 0){
        r = std::min(r, 1 + getMinStepsMemo(n / 2)); // '/2' step
    }

    // Check if the number can be divided by 3
    if(n % 3 == 0){
        r = std::min(r, 1 + getMinStepsMemo(n / 3)); // '/3' step
    }

    // Save the minimum steps
    memo[n] = r; // Save the result

    return r;
}

// Test the execution time of memoization
void testMemoization(int size){

    // Initialize the memoization array
    memo.assign(size + 1, -1);
    memo[1] = 0;

    // Start the timer
    auto begin = std::chrono::steady_clock::now();
    int result = getMinStepsMemo(size);
```

```

    // Stop the timer
    auto end = std::chrono::steady_clock::now();

    // Display the input size
    std::cout << "\nInput Size: " << size << std::endl;

    // Display the execution time
    std::cout << "Execution Time: " << std::chrono::duration_cast<
                 std::chrono::microseconds>(end - begin).count()
                 << " microseconds" << std::endl;

    // Display the minimum steps required
    std::cout << "Minimum Steps Required: "
              << result << std::endl;
}
```

```
67
68  int main(){
69
70      int n;
71
72      // Display the algorithm name
73      std::cout << "\nMemoization:";
74
75      // Get a valid number from the user
76      while(true){
77
78          std::cout << "\n-----------------------------------------"
79                    << std::endl;
80          std::cout << "Enter a number (1-10000): ";
81
82          // Check if the input is valid
83          if(std::cin >> n && n >= 1 && n <= 10000){
84              break; // Valid input
85          }
86
87          // Display an error message
88          std::cout << "Invalid input. Enter a number from 1 to 10000."
89                    << std::endl;
90
91          // Clear the input error and remove the invalid input
92          std::cin.clear();
93          std::cin.ignore(10000, '\n');
94      }
95
96      // Initialize the memoization array
97      memo.assign(n + 1, -1);
98      memo[1] = 0;
99
```

```
100     // Display the result for user input
101     std::cout << "Memoization Result: "
102               << getMinStepsMemo(n) << std::endl;
103
104     // Perform experimental analysis
105     std::cout << "\nExperimental Analysis:";
106     std::cout << "\n-----------------------------------------";
107
108     // Test the algorithm with different input sizes
109     testMemoization(1000);
110     testMemoization(5000);
111     testMemoization(10000);
112
```

```
113     std::cout << std::endl;
114
115     return 0;
116 }
```

Output

```
Memoization:
-----------------------------------------------
Enter a number (1-10000): 100
Memoization Result: 7

Experimental Analysis:
-----------------------------------------------
Input Size: 1000
Execution Time: 48 microseconds
Minimum Steps Required: 9

Input Size: 5000
Execution Time: 310 microseconds
Minimum Steps Required: 13

Input Size: 10000
Execution Time: 263 microseconds
Minimum Steps Required: 14
```

Observation

The Memoization implementation successfully found the minimum steps needed to reach ***1***. When the input was ***100***, it returned ***7 steps***. For the experimental tests, it returned ***9 steps*** for ***1,000, 13 steps*** for ***5,000,*** and ***14 steps*** for ***10,000***. The execution time increased from ***48 microseconds*** to ***310 microseconds***, but later decreased to ***263 microseconds***. This shows that the running time can change during testing.

Theoretical Analysis

The algorithm has a time complexity of ***O(n)*** because each result is calculated and stored only once. Its auxiliary space complexity is also ***O(n)*** because it uses a ***memoization vector*** to save results and ***recursion*** to solve smaller problems. This helps avoid doing the same calculations again and makes the process more efficient.

Experimental Analysis	The recorded execution times were ***48 microseconds*** for ***1,000, 310 microseconds*** for ***5,000,*** and ***263 microseconds*** for ***10,000***. The time did not increase consistently because of possible system activity and measurement differences. Even with this change, the algorithm still found the correct minimum steps and used ***stored results*** to avoid repeated calculations.
Analysis and Comparison	The Memoization implementation successfully found the minimum steps by using ***recursion*** and storing previously calculated results. However, its execution time changed as the input size increased. Compared with ***Bottom-Up Dynamic Programming***, ***Memoization*** may take longer because recursive function calls add extra work. Both methods should still produce the same results for the same input, but their execution times may differ during testing.

Table 12-2. Memoization Implementation

Bottom-Up DP

Run this code and show the output in the table.
(Table 12-3. Bottom-Up Dynamic Programming Implementation)

Screenshot of Code	**My Code: (bottom_up_dynamic_ILOA.cpp)**
	ILOA > bottom_up_dynamic_ILOA.cpp > ...
	1 #include <iostream>
	2 #include <vector>
	3 #include <chrono>
	4 #include <algorithm>
	5
	6 // Store the calculated results
	7 std::vector<int> dp;
	8
	9 // Find the minimum steps to reach 1
	10 int getMinStepsDP(int n){
	11
	12 // Initialize the first result
	13 dp.assign(n + 1, 0);
	14 dp[1] = 0;
	15
	16 // Calculate the minimum steps for each number
	17 for(int i = 2; i <= n; i++){
	18
	19 // Calculate the steps by subtracting 1
	20 dp[i] = 1 + dp[i - 1];
	21

```
        // Check if the number can be divided by 2
        if(i % 2 == 0){
            dp[i] = std::min(dp[i], 1 + dp[i / 2]);
        }

        // Check if the number can be divided by 3
        if(i % 3 == 0){
            dp[i] = std::min(dp[i], 1 + dp[i / 3]);
        }
    }

    return dp[n];
}

// Test the execution time of Bottom-Up DP
void testBottomUpDP(int size){

    // Start the timer
    auto begin = std::chrono::steady_clock::now();
    int result = getMinStepsDP(size);

    // Stop the timer
    auto end = std::chrono::steady_clock::now();

    // Display the input size
    std::cout << "\nInput Size: " << size << std::endl;

    // Display the execution time
    std::cout << "Execution Time: " << std::chrono::duration_cast<
                 std::chrono::microseconds>(end - begin).count()
                 << " microseconds" << std::endl;

```

```
    // Display the minimum steps required
    std::cout << "Minimum Steps Required: "
              << result << std::endl;
}

int main(){

    int n;

    // Display the algorithm name
    std::cout << "\nBottom-Up Dynamic Programming:";

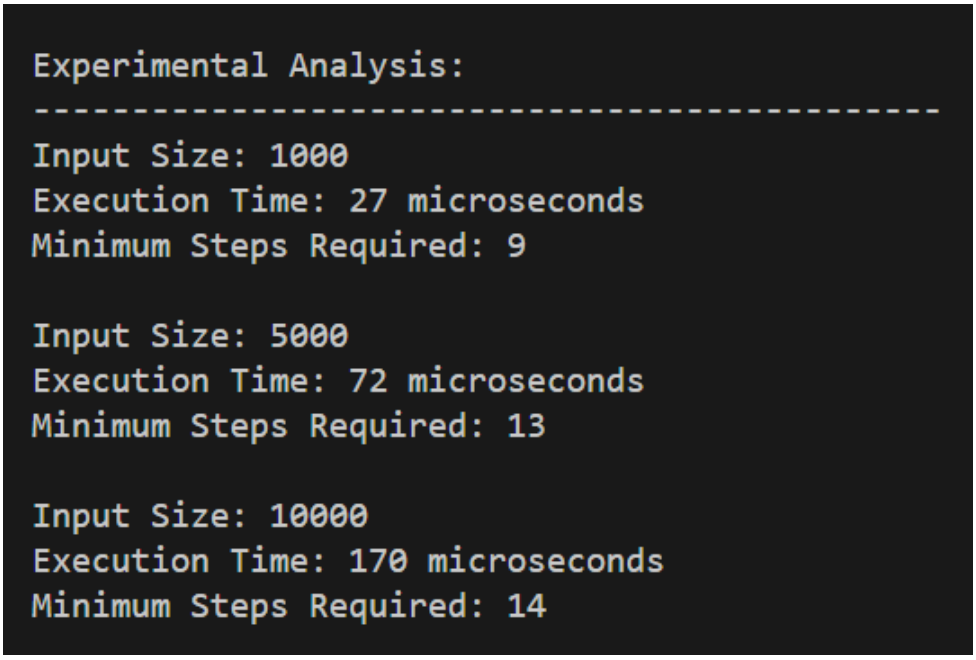
    // Get a valid number from the user
    while(true){

        std::cout << "\n----------------------------------------------"
```

```
70                  << std::endl;
71          std::cout << "Enter a number (1-10000): ";
72
73          // Check if the input is valid
74          if(std::cin >> n && n >= 1 && n <= 10000){
75              break; // Valid input
76          }
77
78          // Display an error message
79          std::cout << "Invalid input. Enter a number from 1 to 10000."
80                    << std::endl;
81
82          // Clear the input error and remove the invalid input
83          std::cin.clear();
84          std::cin.ignore(10000, '\n');
85      }
86
87      // Display the result for user input
88      std::cout << "Bottom-Up DP Result: "
89                << getMinStepsDP(n) << std::endl;
90
91      // Perform experimental analysis
92      std::cout << "\nExperimental Analysis:";
93      std::cout << "\n---------------------------------------------";
94
95      // Test the algorithm with different input sizes
96      testBottomUpDP(1000);
97      testBottomUpDP(5000);
98      testBottomUpDP(10000);
99
100     std::cout << std::endl;
101
102     return 0;
103 }
```

Output

```
Bottom-Up Dynamic Programming:
---------------------------------------------
Enter a number (1-10000): 500
Bottom-Up DP Result: 9
```

	
Observation	The Bottom-Up Dynamic Programming implementation successfully found the minimum steps needed to reach ***1***. When the input was ***500***, it returned ***9 steps***. For the experimental tests, it returned ***9 steps*** for ***1,000, 13 steps*** for ***5,000***, and ***14 steps*** for ***10,000***. The execution time increased from ***27 microseconds*** to ***170 microseconds*** as the input size became larger. This shows that more calculations were needed for bigger inputs.
Theoretical Analysis	The algorithm has a time complexity of ***O(n)*** because it calculates each value from ***2*** up to the given number only once. Its auxiliary space complexity is also ***O(n)*** because it stores the results in a ***DP vector***. It uses ***iteration instead of recursion***, which makes the process more direct and avoids extra recursive function calls.
Experimental Analysis	The recorded execution times were ***27 microseconds*** for ***1,000, 72 microseconds*** for ***5,000,*** and ***170 microseconds*** for ***10,000.*** The results show that the execution time increased as the input size became larger. This happened because more values needed to be calculated and stored. Even with the longer time, the algorithm still gave the ***correct minimum steps*** for all tested inputs.
Analysis and Comparison	The Bottom-Up Dynamic Programming implementation successfully found the minimum steps by using ***iteration*** and storing previously calculated results. Compared with Memoization, it was faster for the input sizes of ***1,000*** and ***5,000*** based on the recorded results. However, the execution times may change depending on the test run. Both methods have ***O(n) time***

	complexity and ***O(n) auxiliary space complexity***, but Bottom-Up DP avoids ***recursive function-call overhead***, which can help it run faster.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell.

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has a given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

Deliverables:

- Pseudocode (+ Solving Problem by Hand)
- Working C++ Code

- Analysis of working code
- Screenshots of demonstration

Pseudocode	**Count Paths Using Recursion:** FUNCTION CountPathsRecursive(matrix, row, col, remaining) SET m = number of rows in matrix SET n = number of columns in matrix IF row >= m OR col >= n THEN RETURN 0 END IF IF remaining < matrix[row][col] THEN RETURN 0 END IF SET remaining = remaining - matrix[row][col] IF row = m - 1 AND col = n - 1 THEN IF remaining = 0 THEN RETURN 1 ELSE RETURN 0 END IF END IF SET right = CountPathsRecursive(matrix, row, col + 1, remaining) SET down = CountPathsRecursive(matrix, row + 1, col, remaining) RETURN right + down END FUNCTION **Count Paths Using Memoization:** FUNCTION CountPathsMemo(matrix, row, col, remaining, memo) SET m = number of rows in matrix SET n = number of columns in matrix

```
    IF row >= m OR col >= n THEN
        RETURN 0
    END IF

    IF remaining < matrix[row][col] THEN
        RETURN 0
    END IF

    SET key = (row, col, remaining)

    IF key exists in memo THEN
        RETURN memo[key]
    END IF

    SET newRemaining = remaining - matrix[row][col]

    IF row = m - 1 AND col = n - 1 THEN
        IF newRemaining = 0 THEN
            SET memo[key] = 1
        ELSE
            SET memo[key] = 0
        END IF

        RETURN memo[key]
    END IF

    SET right = CountPathsMemo(matrix, row, col + 1, newRemaining, memo)
    SET down = CountPathsMemo(matrix, row + 1, col, newRemaining, memo)

    SET memo[key] = right + down

    RETURN memo[key]
END FUNCTION
```

main:

```
START
    CREATE a 4 × 4 matrix
    SET matrix =
        4  7  1  6
```

```
    6  7  3  9
    3  8  1  2
    7  1  7  3

  SET target = 25

  PRINT "Count Paths with a Given Cost:"
  PRINT separator line
  PRINT "Matrix:"
  DISPLAY the matrix
  PRINT "Target Cost: " and target

  PRINT "Experimental Analysis:"
  PRINT separator line

  START recursive timer
  SET recursiveResult = CountPathsRecursive(matrix, 0, 0, target)
  STOP recursive timer
  CALCULATE recursive execution time

  CREATE an empty memoization map

  START memoization timer
  SET memoResult = CountPathsMemo(matrix, 0, 0, target, memo)
  STOP memoization timer
  CALCULATE memoization execution time

  PRINT "Recursive Path Count: " and recursiveResult
  PRINT "Memoized Path Count: " and memoResult
  PRINT recursive execution time
  PRINT memoization execution time

  IF recursiveResult = memoResult THEN
    PRINT "Both results match."
  ELSE
    PRINT "Results do not match."
  END IF
END
```

Solving Problem by Hand	**GIVEN:** ● Matrix size: 4 x 4 ● Starting cell: (0, 0) ● Destination cell: (3, 3) ● Target cost: 25 ● Allowed movements: Right and Down only **SOLUTION:** 1. Start at the first cell with a cost of **4**. 2. Check the possible paths by moving only right or down and adding the costs of all visited cells. 3. No path has a total cost of exactly **25**. However, when the target cost is changed to **26**, two valid paths can be found. **First Path:** Right → Right → Down → Down → Down → Right **4 + 7 + 1 + 3 + 1 + 7 + 3 = 26** **Second Path:** Down → Right → Right → Down → Right → Down **4 + 6 + 7+ 3 + 1 + 2 + 3 = 26** 4. Count the paths that match the target cost. ● For a target cost of 25: **0 paths.** ● For a target cost of 26: **2 paths.** **FINAL ANSWER:** ● The given matrix has **0 paths** with a cost of **25** and **2 paths** with a cost of **26**. Both paths follow the movement rules, and their costs include every visited cell, including the starting and destination cells.

Screenshot of the Code	**My Code: (main_ILOB_C)**

ILOB_C > main_ILOB_C.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <map>
#include <tuple>

// Count paths using recursion
long long countPathsRecursive(
    const std::vector<std::vector<int>>& matrix,
    int row,
    int col,
    int remaining
){

    int m = static_cast<int>(matrix.size());
    int n = static_cast<int>(matrix[0].size());

    // Check if the position is outside the matrix
    if(row >= m || col >= n){
        return 0;
    }

    // Check if the remaining cost is insufficient
    if(remaining < matrix[row][col]){
        return 0;
    }

    // Subtract the current cell cost
    remaining -= matrix[row][col];

```

```cpp
    // Check if the destination is reached
    if(row == m - 1 && col == n - 1){
        return (remaining == 0) ? 1 : 0;
    }

    // Count paths by moving right and down
    return countPathsRecursive(
                matrix, row, col + 1, remaining)
         + countPathsRecursive(
```

```
                matrix, row + 1, col, remaining);
}

// Count paths using memoization
long long countPathsMemo(
    const std::vector<std::vector<int>>& matrix,
    int row,
    int col,
    int remaining,
    std::map<std::tuple<int, int, int>, long long>& memo
){

    int m = static_cast<int>(matrix.size());
    int n = static_cast<int>(matrix[0].size());

    // Check if the position is outside the matrix
    if(row >= m || col >= n){
        return 0;
    }

    // Check if the remaining cost is insufficient
    if(remaining < matrix[row][col]){
        return 0;
    }

    // Check if the result is already stored
    auto key = std::make_tuple(row, col, remaining);
    auto found = memo.find(key);

    if(found != memo.end()){
        return found->second;
    }
```

```

    // Subtract the current cell cost
    int newRemaining = remaining - matrix[row][col];

    // Check if the destination is reached
    if(row == m - 1 && col == n - 1){
        memo[key] = (newRemaining == 0) ? 1 : 0;
        return memo[key];
    }
```

```
81
82      // Count paths by moving right
83      long long right = countPathsMemo(
84          matrix, row, col + 1, newRemaining, memo);
85
86      // Count paths by moving down
87      long long down = countPathsMemo(
88          matrix, row + 1, col, newRemaining, memo);
89
90      // Save the total number of valid paths
91      memo[key] = right + down;
92
93      return memo[key];
94  }
95
96  // Display the matrix
97  void displayMatrix(
98      const std::vector<std::vector<int>>& matrix
99  ){
100
101     for(const auto& row : matrix){
102         for(int value : row){
103             std::cout << value << '\t';
104         }
105
106         std::cout << std::endl;
107     }
108 }
109
```

```
110 // Test the execution time of both algorithms
111 void testPathCount(
112     const std::vector<std::vector<int>>& matrix,
113     int target
114 ){
115
116     // Start the recursive timer
117     auto beginRecursive = std::chrono::steady_clock::now();
118
119     long long recursiveResult =
120         countPathsRecursive(matrix, 0, 0, target);
121
122     // Stop the recursive timer
123     auto endRecursive = std::chrono::steady_clock::now();
```

```
124
125     // Calculate the recursive execution time
126     double recursiveTime =
127         std::chrono::duration<double, std::milli>(
128             endRecursive - beginRecursive).count();
129
130     // Initialize the memoization storage
131     std::map<std::tuple<int, int, int>, long long> memo;
132
133     // Start the memoization timer
134     auto beginMemo = std::chrono::steady_clock::now();
135
136     long long memoResult =
137         countPathsMemo(matrix, 0, 0, target, memo);
138
139     // Stop the memoization timer
140     auto endMemo = std::chrono::steady_clock::now();
141
142     // Calculate the memoization execution time
143     double memoTime =
144         std::chrono::duration<double, std::milli>(
145             endMemo - beginMemo).count();
146
147     // Display the recursive result
148     std::cout << "\nRecursive Path Count: "
149               << recursiveResult << std::endl;
150
151     // Display the memoization result
152     std::cout << "Memoized Path Count: "
153               << memoResult << std::endl;
154
```

```
155     // Display the recursive execution time
156     std::cout << "Recursive Execution Time: "
157               << recursiveTime << " ms" << std::endl;
158
159     // Display the memoization execution time
160     std::cout << "Memoization Execution Time: "
161               << memoTime << " ms" << std::endl;
162
163     // Verify that both results match
164     std::cout << "Verification: "
165               << (recursiveResult == memoResult
166                   ? "Both results match."
```

```
                    : "Results do not match.")
                << std::endl;
}

int main(){

    // Store the matrix from the problem
    std::vector<std::vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int target = 25;

    std::cout << "\nCount Paths with a Given Cost:";

    // Display the matrix
    std::cout << "\n----------------------------------------"
              << std::endl;
    std::cout << "Matrix:" << std::endl << std::endl;
    displayMatrix(matrix);

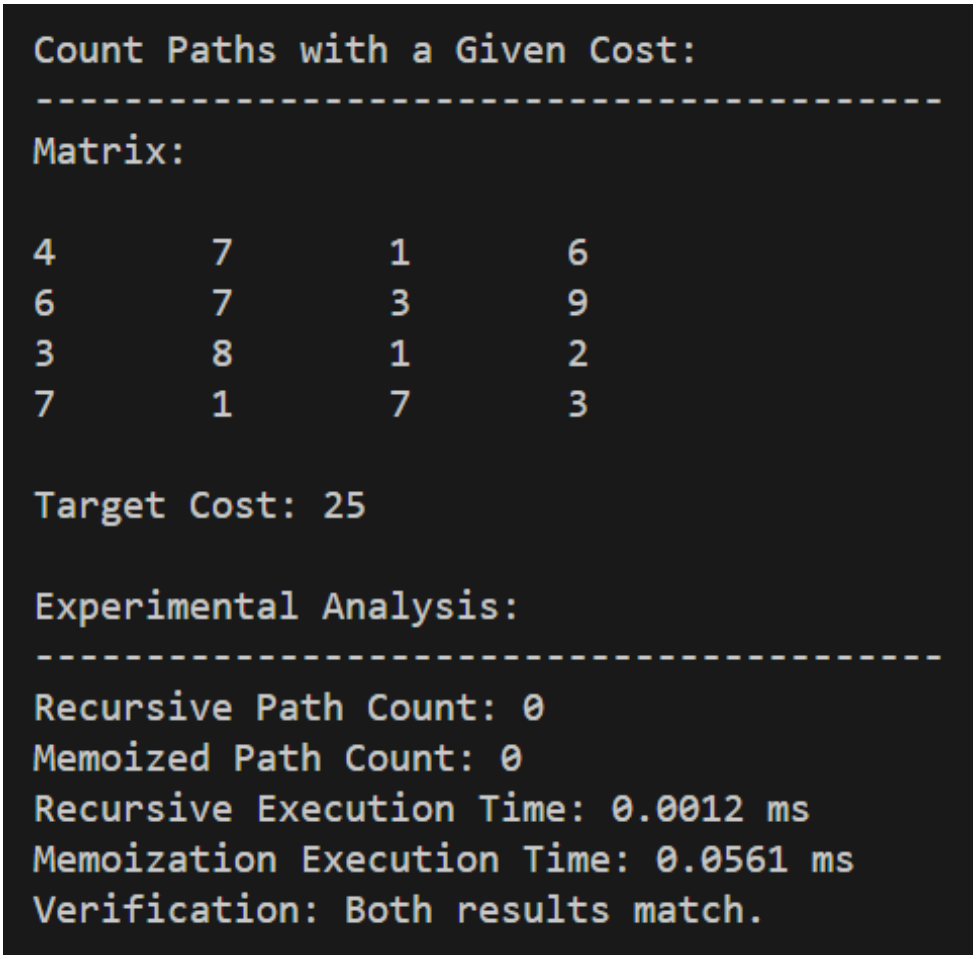
    // Display the target cost
    std::cout << "\nTarget Cost: "
              << target << std::endl;

    // Perform experimental analysis
    std::cout << "\nExperimental Analysis:";
    std::cout << "\n----------------------------------------";

    // Test recursion and memoization
    testPathCount(matrix, target);

    std::cout << std::endl;

    return 0;
}
```

Output	
Observation	The code successfully counted the possible paths using ***Recursion*** and ***Memoization***. Since the target cost was ***25***, both methods returned ***0 paths***, which means there was no valid path that had an exact total cost of ***25***. The recursive method took ***0.0012*** ms, while the memoized method took ***0.0561 ms***. Both methods produced the same result, showing that the code worked correctly, but the recursive method was faster in this small test.

Asajar's Analysis:	
Theoretical Analysis	The ***recursive method*** checks the possible paths by moving right or down until it reaches the destination or finds an invalid path. Its time complexity can become ***exponential*** because it may check many paths repeatedly. Meanwhile, the ***memoization method*** saves calculated results using the row, column, and remaining cost, which helps avoid repeating the same calculations.

Experimental Analysis	The program successfully counted the paths for the target cost of ***25***, and both methods returned ***0 paths***. The recursive method took ***0.0012 ms***, while the memoization method took 0.0561 ms. This shows that the recursive method was faster in this test, while memoization needed extra time to store and manage the results.
Analysis and Comparison	Both methods produced the same ***correct result***, but their execution times were different. The recursive method performed better for this small matrix because there were no valid paths with a cost of 25. However, memoization can become more helpful when there are more repeated calculations in larger problems. Therefore, the better method depends on the ***problem size*** and the number of states that need to be checked.

Godino's Analysis:	
Theoretical Analysis	The ***recursive approach*** solves the problem by checking each possible movement from the current cell, either to the right or downward. As the matrix becomes larger, the number of paths to examine can increase quickly. On the other hand, ***memoization*** keeps previously calculated answers so that the same state does not need to be solved again. This can reduce repeated work, although saving and searching through stored results also requires extra time.
Experimental Analysis	During the test, the recursive and memoized methods both found ***zero valid paths*** for the target cost of 25. The measured execution time was ***0.0012 ms*** for recursion and ***0.0561 ms*** for memoization. The difference suggests that managing stored results took more time than simply exploring the paths in this particular case. Since the matrix is small, these measurements may also be affected by timing variation.
Analysis and Comparison	The test confirms that both approaches can solve the path-counting problem and return matching results. ***Recursion*** was quicker for the given input, while ***memoization*** used an approach designed to prevent repeated calculations. Its extra storage work did not provide a speed advantage in this test. For a larger matrix with many repeated states, memoization may perform better, so both methods should be tested with different input sizes before deciding which one is more efficient overall.

Asajar's Conclusion:

The activity gave a better understanding of how different algorithmic strategies work and how the performance of an algorithm can change depending on the input size and method used. For the lessons learned, Quick Sort uses recursion to divide and sort the elements, Bubble Sort uses brute force by comparing nearby elements, Depth First Search uses backtracking to explore paths, Selection Sort follows a greedy-like approach by selecting the smallest element, and Merge Sort uses divide-and-conquer to divide and combine smaller parts. For the procedure, the Memoization and Bottom-Up Dynamic Programming codes were created and tested using different input sizes to find the minimum steps needed to reach 1. The results showed that both methods produced the same answers, but their execution times were different. For the supplementary activity, Recursion and Memoization were used to count the paths in a matrix with a given cost. Both methods returned zero paths for a target cost of 25, and the recursive method was faster in the recorded test.

One difficulty faced was understanding how each algorithmic strategy works and how to explain its time complexity correctly. Another difficulty was comparing the experimental execution times because they did not always increase consistently, especially in the Memoization test. In the supplementary activity, checking the path costs was also challenging because the two valid paths found had a total cost of 26 instead of 25. Even with these difficulties, the codes produced matching results for the tested methods, and the outputs helped show how each algorithm works. Overall, the activity was done well because the algorithms were implemented and their results were analyzed, but there is still room for improvement in understanding time complexity, checking calculations more carefully, and comparing theoretical and experimental results. More practice with different algorithms and input sizes can help improve the ability to analyze performance and explain the results correctly.

Godino's Conclusion:

This activity helped in understanding how different algorithmic strategies work, such as recursion, brute force, backtracking, greedy, and divide-and-conquer. It also showed how recursion and dynamic programming can be used to solve problems by breaking them into smaller parts. In the minimum steps problem, Memoization and bottom-up dynamic programming both found the correct minimum number of steps, but their execution times were different. In the supplementary activity, the recursive and memoized methods both returned zero valid paths for the target cost of 25. The recursive method was faster in this test because the matrix was small and memoization needed extra time to store and check results. One difficulty was understanding how recursive calls explore different paths and how memoization avoids repeating the same calculations. Overall, the activity was done well because the codes worked correctly and the results were analyzed and compared. However, there is still room for improvement in understanding time complexity, recursion, and how input size affects algorithm performance. More practice with different algorithms and larger test cases can help improve the ability to choose and analyze the most efficient solution.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.